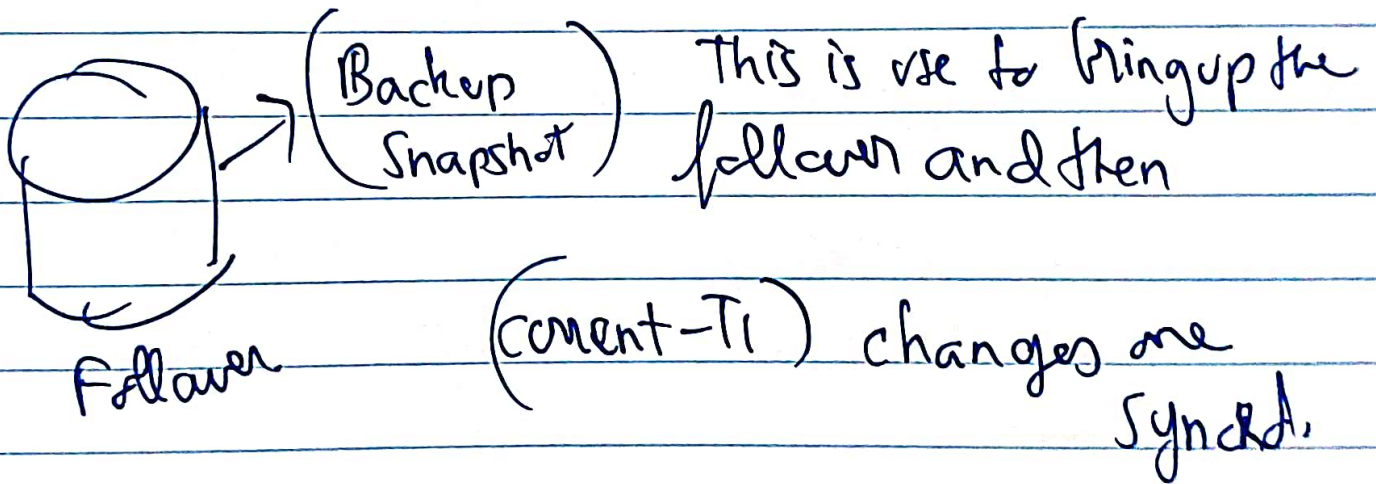


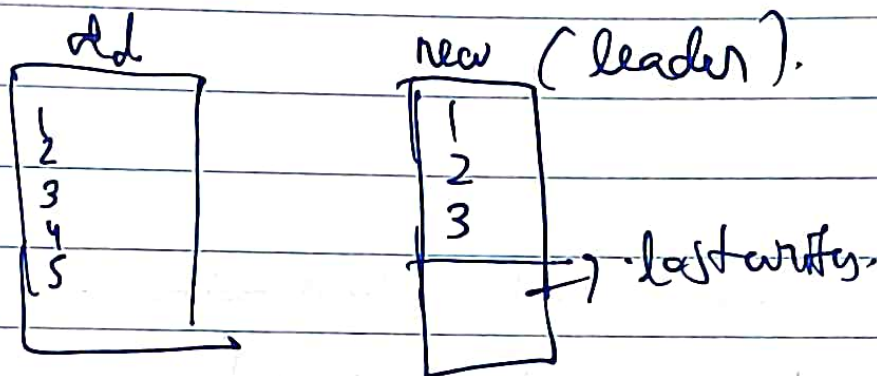
(Chapter 5 Replication)

How A new follower is brought up.

The leader node has a snapshot and that leader node is now ~~sending~~ has a Backup Snapshot till some time T_1 .



In a situation where ~~old writes~~
old leader is ahead, and followers
is there and catching up async
fashion, then if old leader goes down and
new leader comes up, then



Now if old leader comes up, then,
ignore the old writes.

(Replication Logs)

Statement based replication Log

Like (INSERT, DELETE, UPDATE)

(Disadvantages of this is.)

1) if statement has NOW(), RAND(), then
discrepancy.

2) If entry depends on value on some other row, then problem with discrepancy again.

3) There can be trigger on some entry itself

WAL

→ L&B-tree maintain WAL

Disadvantage is if any modification made,

WAL stores at bytes level at OS level which disk block changed with what logs. etc

Hence tightly coupled with OS.

(Logical Row Based Replication)

- inserted row, log contains new values of all columns
- deleted row, unique identifier to mark it deleted
- update → primary key and columns which need updating

Trigger based Replication

→ write in leader leading to high writes in Replicas,

Disadvantage → write amplification.

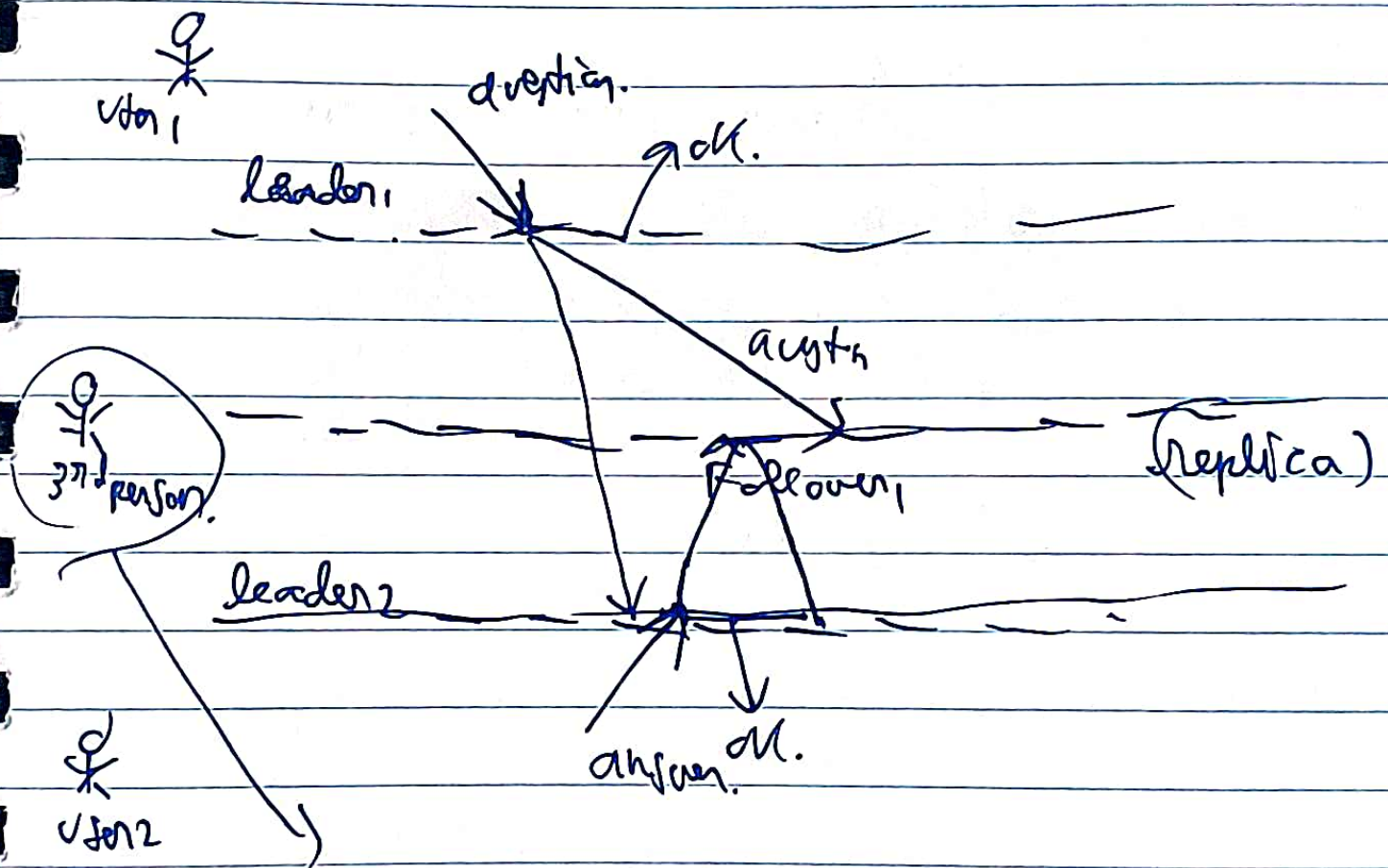
Ways to Solve Replication Lag

1. Read after write consistency (Read Your writes).
user who wrote is directed to the same machine
- * if user wants to see their own profile direct it to their leader. if user wants to see someone else's profile send it to follower.
- * Monitor replication lag
- * Client knows timestamp, they have made the last update, using this we can check replica has the update or not

(Monotonic Reads)

Guarantee that time never goes backwards. Once he read the new data, he will not read old data.

Consistent Prefix Read



3rd person will see the ~~question~~ answer before the question,
For that we have to enable causal consistency.

Causal Consistency says that sequence of writes will never be destroyed

(Multi-Leader Architecture)

→ ~~Single-leader~~ Since in multi-leader + async writes there conflicts.

So few ways to resolve conflicts.

→ direct writes to particular replica, like in ~~google~~ google does direct user to only one replica, have a

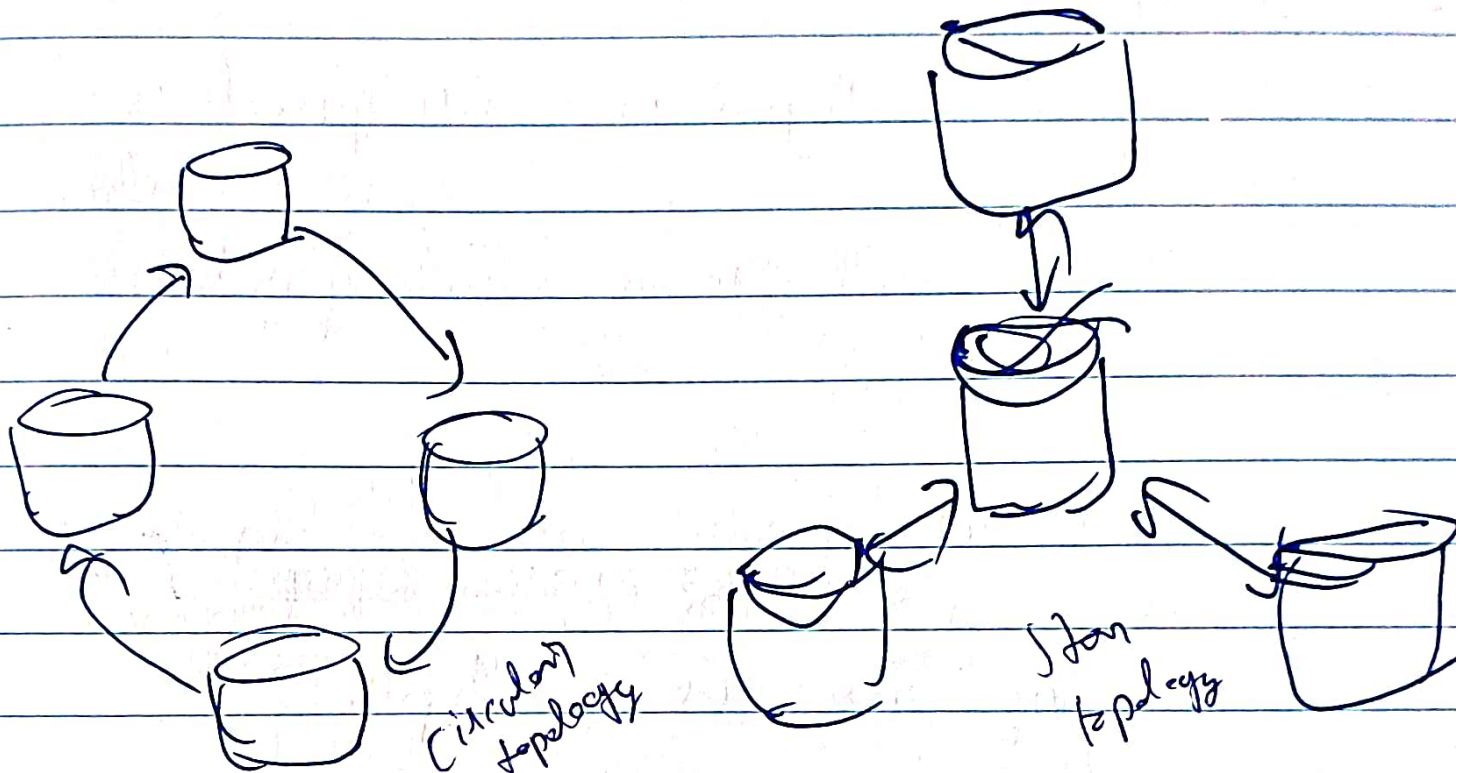
[user → replica] map

But if leader fails complete point of conflict avoidance fails.

Converge Conflicts →

like by using snowflake id, and monotonically increasing and

unique so we can use that
(last write wins).



(Amazon's Dynamo DB) (Read Repair method)

→ During Read just check which replica has stale value [we query multiple replicas]. And then update it with latest version

→ Anti Entropy Process → Background thread runs that constantly looks for difference in data b/w replicas.

Quorums.

$(w + r)$

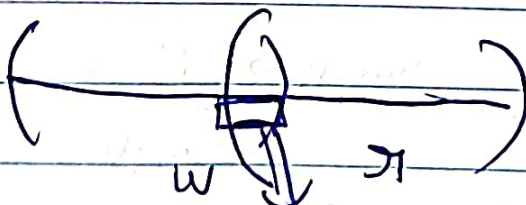
$w < n$ Still process writes if node is
unavailable

$r < n$ still process reads if node is
unavailable

read and write queries are sent ~~open~~ in parallel, the parameters w and r determine how many nodes we wait for.

if less than w or r nodes available we give error

$(w + r < n)$ { Compromise on consistency
but very high availability }

$(w + r > n)$ 

only small overlap.
What might happen is old values might

get read due to stale values in majority of n nodes.

Due to quorum, a small group of nodes going down or a single node failing does not lead to entire trigger of failover.

~~***~~
Floppy Quorum $\rightarrow$ (Cassandra does not use it).

During writes lets say you had some writes w nodes on which you can write, but lets say ~~these~~ ~~or~~ ~~for~~ $(w/2)$ of those w nodes are down, then what happens.

Floppy quorum says, write the writes to some placeholder node for now and when network comes in, then write to ~~other~~ ~~network~~ that quorum node.

(Increases write availability, [Hinted Handoff].
But read consistency or durability is compromised).